

Session-5 - Higher-Order Functions :

5a-Squares of a Sorted Array :

Program :

```
class Solution {  
    public int[] sortedSquares(int[] nums) {  
        int n = nums.length;  
        int[] result = new int[n];  
        int left = 0, right = n - 1;  
        int pos = n - 1;  
        while (left <= right) {  
            int leftSquare = nums[left] * nums[left];  
            int rightSquare = nums[right] * nums[right];  
            if (leftSquare > rightSquare) {  
                result[pos] = leftSquare;  
                left++;  
            } else {  
                result[pos] = rightSquare;  
                right--;  
            }  
            pos--;  
        }  
        return result;  
    }  
}
```

Output :

Accepted 137 / 137 testcases passed
Praveen_1437 submitted at Sep 14, 2026 22:41

Runtime
1 ms | Beats 100.00%
Memory
47.15 MB | Beats 75.50%

Code | Java

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int n = nums.length;
4         int[] result = new int[n];
5         int left = 0, right = n - 1;
6         int pos = n - 1;
7
8         while (left <= right) {
9             int leftSquare = nums[left] * nums[left];
10            int rightSquare = nums[right] * nums[right];
11
12            if (leftSquare > rightSquare) {
13                result[pos] = leftSquare;
14                left++;
15            } else {
16                result[pos] = rightSquare;
17                right--;
18            }
19            pos--;
20        }
21        return result;
22    }
23 }
```

Test Result
Accepted Runtime: 0 ms

Case 1 Case 2

Input
nums =
[-4, -1, 0, 3, 10]

Output
[0, 1, 9, 16, 100]

Expected
[0, 1, 9, 16, 100]

5b: Find Pivot Index :

Program :

```
class Solution {  
    public int pivotIndex(int[] nums) {  
        int totalSum = 0;  
        int leftSum = 0;  
        for (int ele : nums) {  
            totalSum += ele;  
        }  
        for (int i = 0; i < nums.length; i++) {  
            if (leftSum * 2 == totalSum - nums[i]) {  
                return i;  
            }  
            leftSum += nums[i];  
        }  
        return -1;  
    }  
}
```

Output :

The screenshot displays a coding interface for the problem "724. Find Pivot Index". The problem description states: "Given an array of integers `nums`, calculate the **pivot index** of this array. The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right. If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array. Return the **leftmost pivot index**. If no such index exists, return `-1`."

Example 1:
Input: `nums = [1,7,3,6,5,6]`
Output: `3`
Explanation: The pivot index is 3. Left sum = `nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11`. Right sum = `nums[4] + nums[5] = 5 + 6 = 11`.

Example 2:
Input: `nums = [1,2,3]`
Output: `-1`
Explanation: There is no index that satisfies the conditions in the problem statement.

Example 3:

The solution code is shown in the editor, matching the program provided in the previous blocks. The test results section shows three cases (Case 1, Case 2, Case 3) all marked as "Accepted" with a runtime of 0 ms. The input for the test cases is `nums = [1,7,3,6,5,6]` and the output is `3`.